

Herald



Herald — commons_watch Module Guide (Operator / Developer)

Field	Value
Revision	1
Created	2026-05-30
Last modified	2026-05-30
Status	active
Status summary	Nano-detail per-module reference for <code>commons_watch</code> — the <code>fsnotify</code> + WAL-poll file watcher that sits over the workable-items SQLite SSoT and its Markdown trackers and emits debounced, coalesced change Events. Documents the <code>Options/New/Events/Start/Close</code> API, the <code>debounce-coalesce + dedup</code> behaviour, WHY the WAL-poll fallback exists (SQLite WAL writes hit the <code>-wal</code> sidecar and may not raise a reliable <code>fsnotify</code> Write on the main <code>.db</code>), and the leak-free <code>ctx-cancel</code> shutdown. ANTI-BLUFF: every section documents only what the source under <code>commons_watch/</code> actually does as of this revision.
Issues	(none specific to this guide)
Continuation	bump when the watcher gains a typed Event payload (currently <code>Op</code> is just <code>"fsnotify" / "poll"</code>), and when the live <code>pherald_watch</code> wiring documented in <code>WORKABLE_ITEMS_INTEGRATION.md</code> lands operator-supplied evidence.

Table of contents

- [§1. Overview](#)
 - [§2. The API](#)
 - [§3. Why the WAL-poll fallback exists](#)
 - [§4. Debounce, coalesce, and dedup](#)
 - [§5. Graceful shutdown and leak-freedom](#)
 - [§6. Usage examples](#)
 - [§7. Testing notes](#)
 - [§8. References](#)
-

§1. Overview

`commons_watch` (Go package `commons_watch`, module path `github.com/vasic-digital/herald/commons_watch`) watches a set of files — a SQLite `.db` single-source-of-truth plus its Markdown trackers — and emits **coalesced** change Events. It is the trigger layer that tells the workflow bridge "the SSoT changed, recompute the diff." See `docs/guides/COMMONS_WORKABLE.md` for the store it watches over and `docs/guides/WORKABLE_ITEMS_INTEGRATION.md` for the end-to-end flow.

It combines **two** detection mechanisms (both documented in the package doc at the top of `watch.go`):

1. **fsnotify** — kernel-level `inotify` / `kqueue` / `FSEvents` notifications on the watched files **and** their parent directories (so atomic rename-replace writes are seen).
2. **A WAL-poll fallback** (enabled when `PollInterval > 0`) — a periodic (`mtime`, `size`) `stat` of each watched path that synthesizes an Event when either changes. This is the backstop for SQLite WAL-mode writes that never raise a reliable `fsnotify` Write on the main `.db` inode (see §3).

Both mechanisms funnel through one `schedule` path, so a change seen by **both** is emitted **once** per debounce window.

The only third-party dependency is `github.com/fsnotify/fsnotify v1.8.0` (per `commons_watch/go.mod`); everything else is the Go standard library.

§2. The API

§2.1 Event

```
type Event struct {
    Path string // the watched file path that changed
    Op    string // originating signal: "fsnotify" or "poll"
}
```

`Op` is a short provenance string only — it is `"fsnotify"` when the kernel notification fired first and `"poll"` when the WAL-poll loop detected the change first. There is no richer payload today.

§2.2 Options

```
type Options struct {
    Paths          []string // the set of file paths to watch
    Debounce       time.Duration // coalescing window; defaults to
    200ms when zero
    PollInterval   time.Duration // > 0 enables the WAL-poll
    fallback; 0 disables it
}
```

Field	Default	Meaning
<code>Paths</code>	—	The exact file paths to watch. Membership is exact-match (<code>ev.Name</code> must equal a watched path).

Field	Default	Meaning
Debounce	200ms (defaultDebounce) when <code><= 0</code>	Window within which rapid successive changes on the same path coalesce into one Event.
PollInterval	0 (disabled)	When <code>> 0</code> , each path's (<code>mtime</code> , <code>size</code>) is sampled at this interval and a synthetic Event is emitted on change.

§2.3 Constructor and methods

Function / method	Signature	Behaviour
New	New(opts Options) (*Watcher, error)	Constructs the Watcher. Applies the 200ms debounce default. Creates the fsnotify watcher and registers each path AND its parent directory (parent registration is best-effort-on-the-file, mandatory-on-the-dir; a dir-add failure closes fsnotify and returns the error). Files that don't exist yet are added best-effort.
(*Watcher) Events	Events() <-chan Event	Returns the read-only channel on which coalesced Events arrive (buffered, capacity 64).
(*Watcher) Start	Start(ctx context.Context) error	Runs the watch loops until <code>ctx</code> is cancelled; returns <code>ctx.Err()</code> (or <code>nil</code> when the fsnotify channels close). Blocking — run it in a goroutine.
(*Watcher) Close	Close() error	Releases the underlying fsnotify watcher. Idempotent (<code>sync.Once</code>) — safe to call multiple times.

§2.4 Why both the file and its directory are watched

New adds the file path itself **and** its parent directory to fsnotify. Editors and SQLite frequently write via **atomic rename-replace** (write a temp file, then `rename` over the target). A watch on the original inode can miss that, but a watch on the **directory** sees the rename. Registering both is what makes editor saves and atomic DB swaps observable. The directory `dir` is de-duplicated across paths (`seenDir` map) so a directory shared by several watched files is only added once.

§3. Why the WAL-poll fallback exists

This is the load-bearing design decision in the module, and it is documented verbatim in the package doc.

`commons_workable` opens the SSOT in **WAL journal mode** (`PRAGMA journal_mode=WAL` — see `docs/guides/COMMONS_WORKABLE.md` §2.5). In WAL mode:

- Logical DB mutations are first written to the `<db>-wal` **sidecar** file, and
- Only **periodically checkpointed** back into the main `.db` file.

Consequently, **a logical DB write may produce NO reliable fsnotify Write event on the main .db inode within a useful window** — the bytes landed in the `-wal` sidecar, not the `.db`. An

fsnotify-only watcher on `foo.db` can therefore sit silent while the database is, in fact, being mutated.

The poll loop is the backstop. When `PollInterval > 0`:

- `pollLoop` seeds a baseline (`mtime`, `size`) snapshot for every watched path at startup (so the **first observed change**, not the initial state, fires).
- On each tick it re-stats every path (`statOf`) and, if (`mtime`, `size`) differs (`changed`), schedules a synthetic "poll" Event.
- A missing file yields `known=false`; transitions in existence (`present`☐`absent`) are themselves treated as changes, while staying-absent is not.

To actually catch WAL writes, callers add the `-wal` sidecar path to `Options.Paths` and enable polling. `TestWatch_PollFallbackDetectsSidecar` proves this: it watches both `ssot.db` and `ssot.db-wal`, mutates only the sidecar (bumping its `mtime` forward), and asserts a poll-sourced Event fires for the sidecar — independent of fsnotify.

Operator guidance. For a WAL-mode SSOT, watch both the `.db` and the `.db-wal` sidecar and set a `PollInterval` (e.g. 100ms–1s). fsnotify alone on the `.db` is not sufficient to see in-flight WAL writes before a checkpoint.

§4. Debounce, coalesce, and dedup

Both the fsnotify and poll paths call the single private `schedule(ctx, path, op)`, which provides three guarantees within one debounce window:

1. **Dedup against recent emit.** If an Event for this path was already emitted within the last Debounce window (`lastEmit[path]`), the new signal is dropped. This is what makes a change seen by *both* fsnotify and the poll loop emit only once.
2. **Coalesce pending bursts.** If a debounce timer is already pending for this path, a rapid follow-up signal does nothing — it is folded into the already-scheduled emit.
3. **Delayed single emit.** Otherwise a `time.AfterFunc(Debounce, ...)` timer is armed; when it fires it clears the pending entry, records `lastEmit[path] = now`, and sends one Event on the channel (or bails if `ctx` is done / the watcher closed).

`TestWatch_DebounceCoalesces` proves the coalesce: 5 rapid writes within the window produce **exactly one** Event. `TestWatch_EmitsOnModify` proves a single modify produces at least one Event with the correct path.

The first observed signal's `Op` is the one carried on the emitted Event (captured as `op0` when the timer is armed).

§5. Graceful shutdown and leak-freedom

`Start` runs the fsnotify event loop directly and, when `PollInterval > 0`, the poll loop as a child goroutine tracked by a `sync.WaitGroup`. Shutdown is driven by `ctx` **cancellation**:

- On `<-ctx.Done()` (or either fsnotify channel closing), `Start` calls `cancelPending()` to `Stop()` and drop every in-flight debounce timer, then `wg.Wait()`s for the poll goroutine to exit, then returns `ctx.Err()` (or `nil`).
- The poll loop selects on `<-ctx.Done()` and returns immediately on cancel.

- The debounce timer's send is itself guarded by a `select` over `ctx.Done()` and the closed channel, so a timer firing during teardown cannot block forever on a full/abandoned channel.
- `Close()` (idempotent via `sync.Once`) closes the internal closed channel and the fsnotify watcher.

`TestWatch_CancelNoGoroutineLeak` is the proof: it snapshots `runtime.NumGoroutine()` before, starts a watcher with both fsnotify and polling active, cancels the context, asserts `Start` returns within 2s, calls `Close()`, and asserts the post-settle goroutine delta is ≤ 1 . So a cancelled watcher leaves no leaked goroutines.

Lifecycle order. Cancel the context to stop the loops, then call `Close()` to release the fsnotify handle. Both orders are safe (`Close` is idempotent), but cancel-then-close is the pattern the tests exercise.

§6. Usage examples

§6.1 Watch a WAL-mode SSoT plus its trackers

```
import (
    "context"
    "time"
    commons_watch "github.com/vasic-digital/herald/commons_watch"
)

w, err := commons_watch.New(commons_watch.Options{
    Paths: []string{
        "/path/workable_items.db",
        "/path/workable_items.db-wal", // the WAL sidecar — see §3
        "/path/Issues.md",
        "/path/Fixed.md",
    },
    Debounce: 200 * time.Millisecond,
    PollInterval: 250 *
        time.Millisecond, // enables the WAL-poll fallback
})
if err != nil { /* handle */ }
defer w.Close()

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

go func() { _ = w.Start(ctx) }()

for ev := range w.Events() {
    // ev.Path changed; ev.Op is "fsnotify" or "poll".
    // Recompute the workable.Diff and fan out notifications.
}
```

§6.2 fsnotify-only (no polling)

```
// Leave PollInterval at its zero value to disable the poll loop
entirely.
w, _ := commons_watch.New(commons_watch.Options{
    Paths:    []string{"/path/Issues.md"},
    Debounce: 200 * time.Millisecond,
})
```

This is appropriate for plain Markdown trackers where editor saves do raise fsnotify events; it is **not** sufficient for a WAL-mode .db (see §3).

§7. Testing notes

Tests live in `commons_watch/watch_test.go` and run with no external services (real temp files under `t.TempDir()`):

```
go test -race -count=1 ./commons_watch/...
```

Test

`TestWatch_EmitsOnModify`

`TestWatch_DebounceCoalesces`

`TestWatch_PollFallbackDetectsSidecar`

`TestWatch_CancelNoGoroutineLeak`

Proves

A watched-file modify emits ≥ 1 Event with the right Path.

5 rapid writes within the window → **exactly 1** coalesced Event.

The poll loop fires for a -wal sidecar change **independent of fsnotify** (the WAL rationale, end-to-end).

ctx-cancel returns `Start` cleanly with goroutine delta ≤ 1 (leak-free).

Anti-bluff observations worth preserving when editing tests:

- The poll test bumps the sidecar's mtime forward with `os.Chtimes` to guarantee the stat-poll definitely observes the change — it tests real filesystem behaviour, not a mock.
- The leak test uses settle-delays before/after to avoid counting unrelated runtime goroutines; keep them if you touch the timing.
- Events are drained with a windowed `collectEvents` helper rather than a single receive, so coalesce/dedup counts are asserted exactly.

§8. References

- Source: `commons_watch/watch.go` and `commons_watch/watch_test.go`.
- Package doc: the comment block at the top of `watch.go` (the two-mechanism + WAL rationale).
- Watched store: `commons_workable` — the SQLite SSoT and its WAL journal mode (`docs/guides/COMMONS_WORKABLE.md`, esp. §2.5).
- Integration: `docs/guides/WORKABLE_ITEMS_INTEGRATION.md` (the pherald watch flow this watcher triggers).
- Dependency: `github.com/fsnotify/fsnotify v1.8.0` (`commons_watch/go.mod`).

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on beyond the vendored `fsnotify` already pinned in `commons_watch/go.mod`. All behavioural claims are grounded in the cited source files as of 2026-05-30.

Verified 2026-05-30: internal doc — no external online sources. Behavioural claims derive from `commons_watch/watch.go` + `commons_watch/watch_test.go` (read 2026-05-30); the only third-party dependency is `github.com/fsnotify/fsnotify v1.8.0`, pinned in `commons_watch/go.mod` (no online-doc cross-reference required — the API surface used is the vendored, version-pinned one). Re-verify on an `fsnotify` major-version bump or a `commons_watch` API change.